

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework Assignment 4

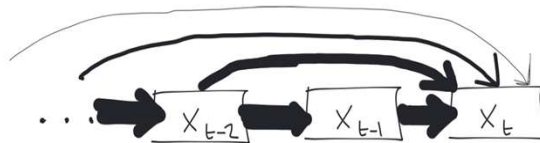
- Due: **11:59 PM** on **Monday, April 27**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

Lecture 24

- Exponential Smoothing
 - Triple Exponential Smoothing
- Clustering
 - Introduction to clustering
 - Distance (or similarity) measures
 - Hierarchical clustering

Previously... Exponential smoothing

- Exponential smoothing mitigates the shortcomings of the rolling window and weighted moving averages.
- Intuition: Instead of using simply a subset of the data, we will use all past observations.
 - The contributions (weights) associated with each point decays exponentially as the observations “get older”.



- This simple concept is the basis for some of the most successful forecasting methods.
 - One such method is exponential smoothing.

Previously... Single exponential smoothing

- Computes smoothed data s_t at time t using the observed value v_t and smoothed value from previous time step (s_{t-1})

$$s_t = \alpha \cdot v_t + (1 - \alpha)s_{t-1} \quad \text{where } 0 \leq \alpha \leq 1$$

$\vdots$

$$= \alpha[v_t + (1 - \alpha)v_{t-1} + (1 - \alpha)^2v_{t-2} + \dots + (1 - \alpha)^{t-1}v_1] + (1 - \alpha)^t v_0$$

- Forecasting:

$$F_{t+1} = s_t$$

$\vdots$

$$= F_t + \alpha(v_t - F_t)$$

$$= F_t + \alpha E$$

Previously... Double exponential smoothing

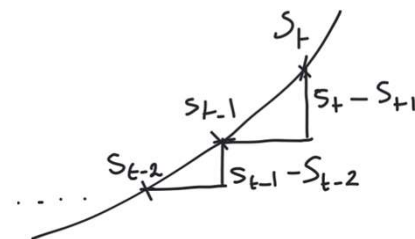
- Double exponential smoothing add the information about the trend.
 - Consider the trend as the exponentially weighted changes since $t = 0$.
 - The most recent change gets weight of β .
- We can infer the trend r_t using the same approach used to compute s_t :

$$r_t = \beta(s_t - s_{t-1}) + (1 - \beta)r_{t-1}$$

- Forecasting:

$$F_{t+1} = s_t + r_t$$

$$\text{where } s_t = \alpha \cdot v_t + (1 - \alpha)(s_{t-1} + r_{t-1})$$



Triple exponential smoothing

- The assumption here is that there is seasonality in the data.
 - We need another “quantity” to capture the seasonality.
 - Similar to how we derived a quantity to capture the trend.
- The idea is to capture seasonality by using the data across past seasons at the same time in the period.
 - For instance, in time series with a yearly seasonality, the seasonality components for March would be exponentially smoothed with only data points from March in previous years.
- A non-parametric way to account for seasonality.

Triple exponential smoothing (cont'd.)

- The seasonal component in triple exponential smoothing is updated as:

$$p_t = \gamma(v_t - s_t) + (1 - \gamma)p_{t-k}$$

Handwritten notes: "current signal including trend comp." with a red arrow pointing to $v_t - s_t$; "deviation by" with a blue arrow pointing to s_t ; "remove both level (signal) + trend" with a blue arrow pointing to p_{t-k} .

where

- k is the length of the seasonal period.
 - This is a parameter of the model; can be derived based on domain knowledge or using methods like ACF, or by finding the value that best fits the data.
- γ is the smoothing factor for the seasonal component.

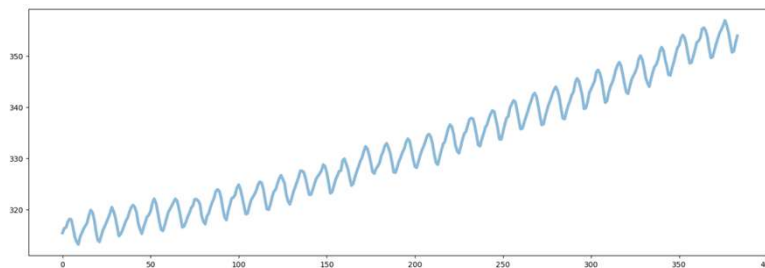
Triple exponential smoothing: Remarks

- The concept here is similar to what we have done in the past.
- Seasonality component p_i is weighted using the most recent observed point exactly one period, k , ago.
- The function for predicting future values is computed as:

$$F_{t+1} = s_t + r_t + p_{t+1-k}$$

Testing the exponential smoothing

- Does the approach pass muster with the CO₂ dataset we manually investigated?
 - The data exhibits clear trend and seasonality.



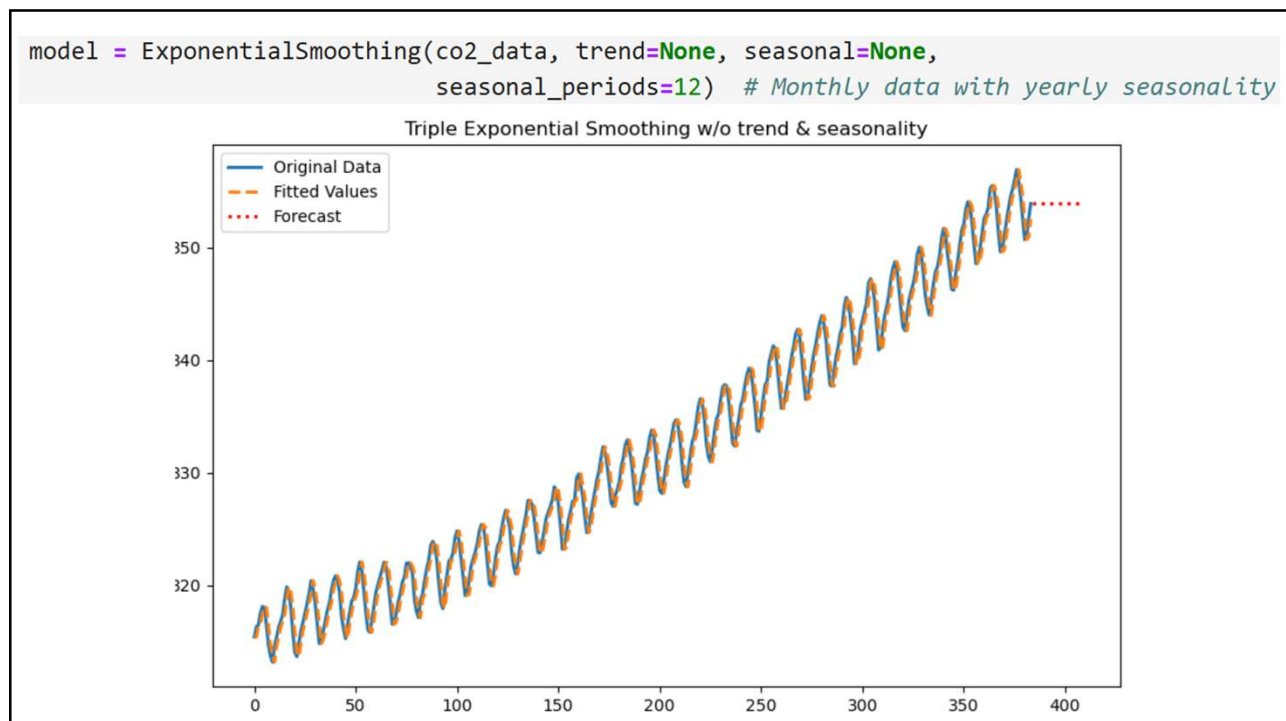
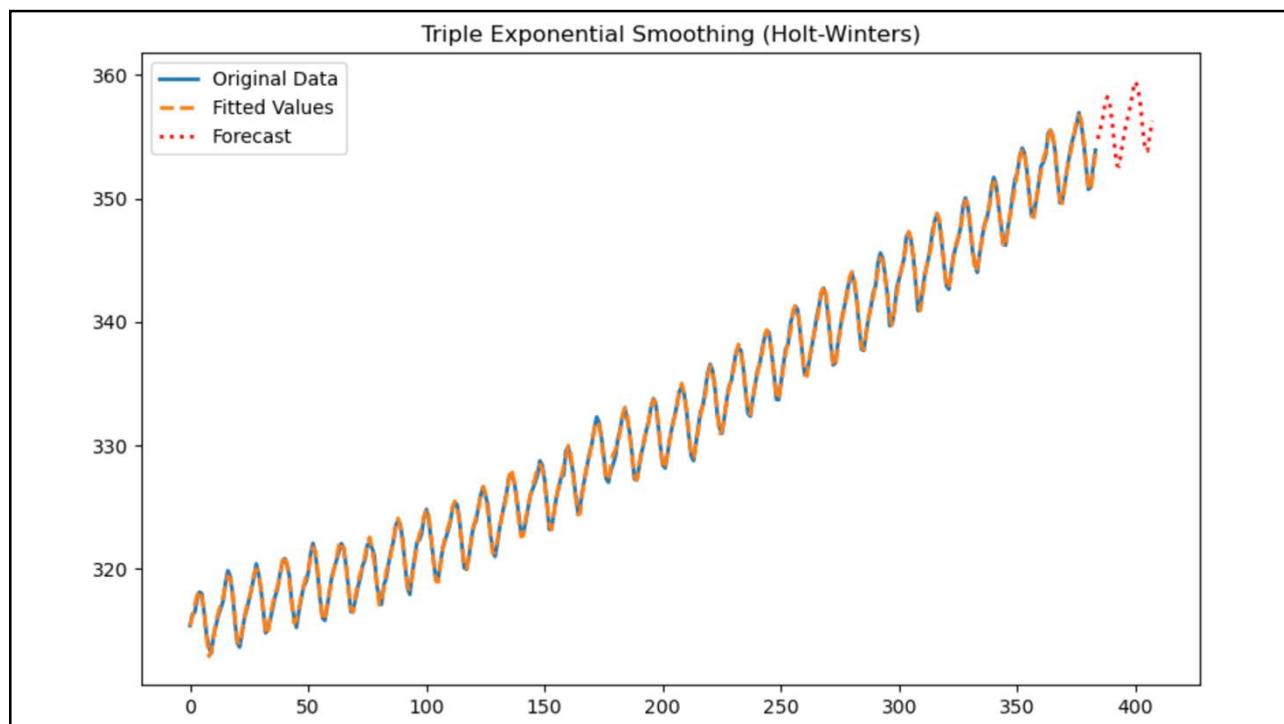
- Here we use statsmodels package.

```
import pandas as pd
```

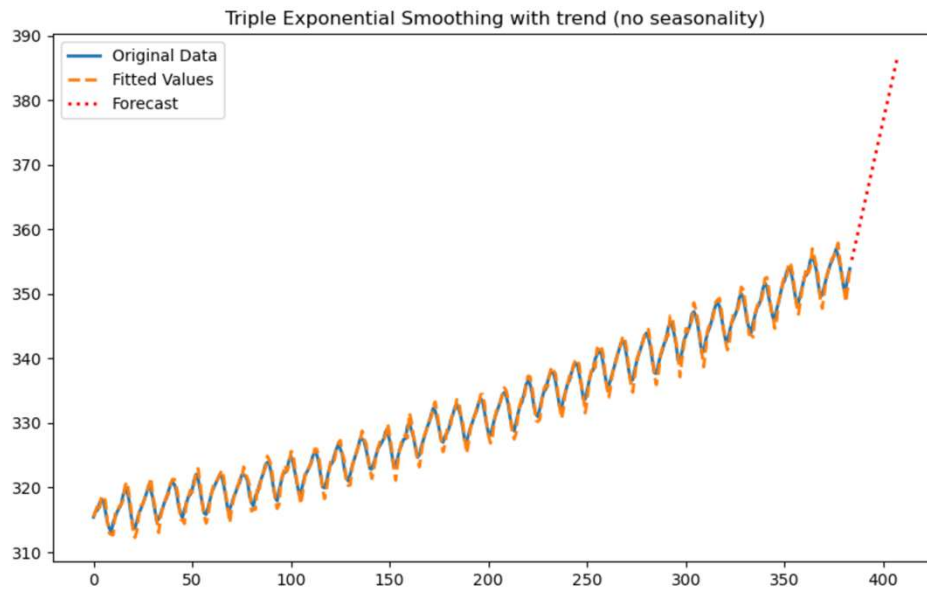
```
co2_data = pd.read_csv("data/carbon_dioxide.txt", names=["co2_val"])  
co2_data.co2_val = co2_data.co2_val.astype('float64')  
co2_data.head()
```

	co2_val
0	315.42
1	316.32
2	316.49
3	317.56
4	318.13

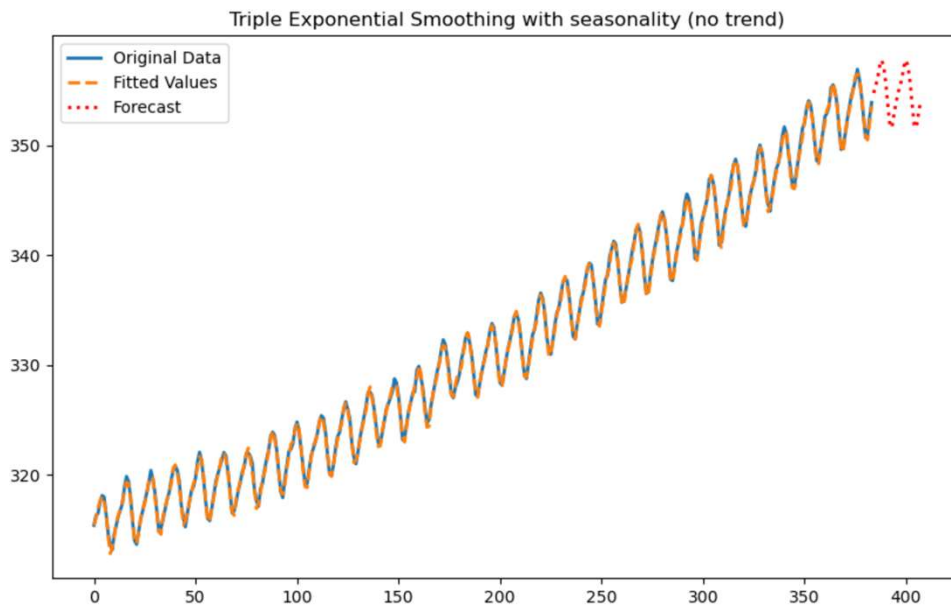
```
from statsmodels.tsa.holtwinters import ExponentialSmoothing  
  
model = ExponentialSmoothing(co2_data, trend='add', seasonal='add',  
                             seasonal_periods=12) # Monthly data with yearly seasonality  
  
tes = model.fit() # The three parameters are optimized automatically  
  
# Make predictions  
forecast = tes.forecast(24)  
  
# Plot the results  
plt.figure(figsize=(10, 6))  
plt.plot(co2_data, lw=2, label='Original Data')  
plt.plot(tes.fittedvalues, lw=2, label='Fitted Values', linestyle='--')  
plt.plot(forecast, lw=2, label='Forecast', linestyle=':', color='red')  
plt.legend(loc='best')  
plt.title('Triple Exponential Smoothing (Holt-Winters)');
```



```
model = ExponentialSmoothing(co2_data, trend="add", seasonal=None,
                             seasonal_periods=12) # Monthly data with yearly seasonality
```



```
model = ExponentialSmoothing(co2_data, trend=None, seasonal="add",
                             seasonal_periods=12) # Monthly data with yearly seasonality
```



Clustering

What is clustering?

- Clustering analysis, or clustering, is a computational approach to uncover structure in a dataset.
 - Finding groups of points within a dataset that are grouped or lumped together.
- It is a machine learning task.
 - Clustering is an *unsupervised* method because no labels or examples are provided to indicate which observations should belong together.
 - In contrast, supervised learning uses labeled data with a response variable y to learn how to assign observations to predefined groups (e.g., classifying emails as spam or ham/legit).

What is a cluster?

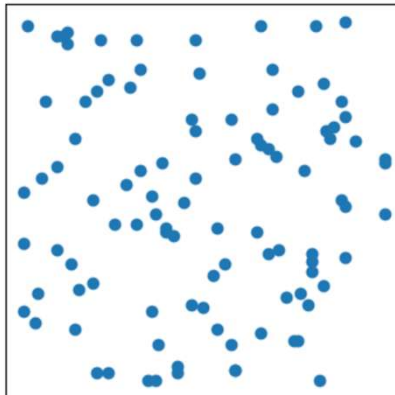
- Descriptions such as “groups of points that are similar” or “close to each other”.
- The notion of a “cluster” is not well defined.
 - Clusters may be at times subjective.
 - The concept of boundary around a set of similar points is subjective.
 - Once we see a cluster we “kind of” know what it is.

similarity? What is the definition of a cluster? Fig. 2 shows that clusters can differ in terms of their *shape*, *size*, and *density*. The presence of noise in the data makes the detection of the clusters even more difficult. An ideal cluster can be defined as a set of points that is *compact* and *isolated*. In reality, a cluster is a subjective entity that is in the eye of the beholder and whose significance and interpretation requires domain knowledge. But, while humans are excellent cluster seekers in two and possibly three dimensions, we need automatic algorithms for high-dimensional data. It is this challenge along with the unknown number of clusters for the given data that has resulted in thousands of clustering algorithms that have been published and that continue to appear.

Data clustering: 50 years beyond K-means, Jain 2009

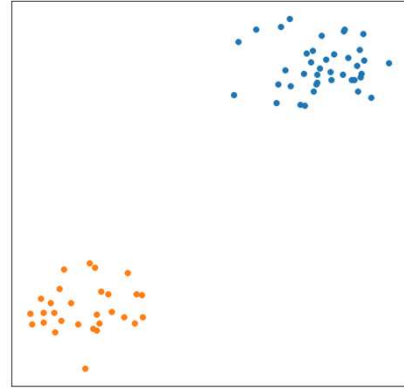
Question

- Is a cluster simply a group of points that are close?



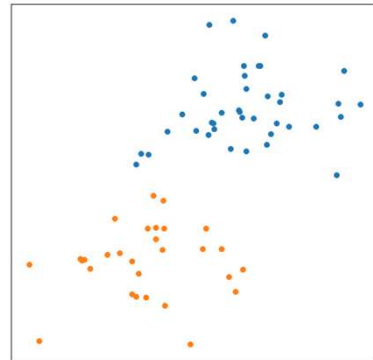
What is a cluster? (cont'd.)

- We can define a cluster as a contiguous region of high data point density.
 - Points in the region are close to each other.
 - Areas contain more than just a few points that seem to occur randomly together.
- Regions are separated by areas of lower point density.
 - Points across regions are dissimilar.



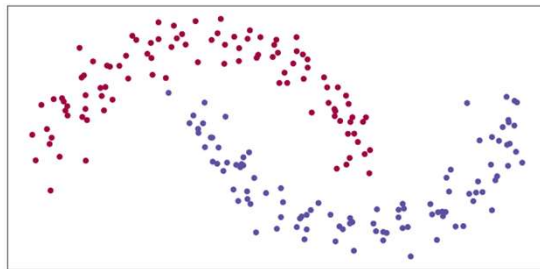
What is a cluster? (cont'd.)

- Contiguous regions of high data point density need not be highly dissimilar.
 - On average, points across regions are dissimilar.
 - Some points may be closer than others.



Data shapes

- Clusters may have complicated shapes.
 - While simple or well-behaved clusters are orderly, real clusters can be very messy.
 - Some clusters do not subscribe to the definitions earlier.
 - Such clusters may challenge simple solutions.



Applications of clustering

- Clustering can be applied in a variety of contexts, from genetics and genomics to user input consolidation and investment.
- Definition of the clusters across domains is the same.
 - Regions of high-density points where points characterized by high similarity within regions and low similarity across regions.
- Naturally, the concept of similarity and distance between points is domain-specific.
 - User similarity (e.g., Netflix): Based on agreement in ratings or overlap in watched movies.
 - Asset similarity (e.g., Apple vs. Qualcomm): Based on how their stock prices move together; useful for portfolio diversification.

Example: Data reconciliation

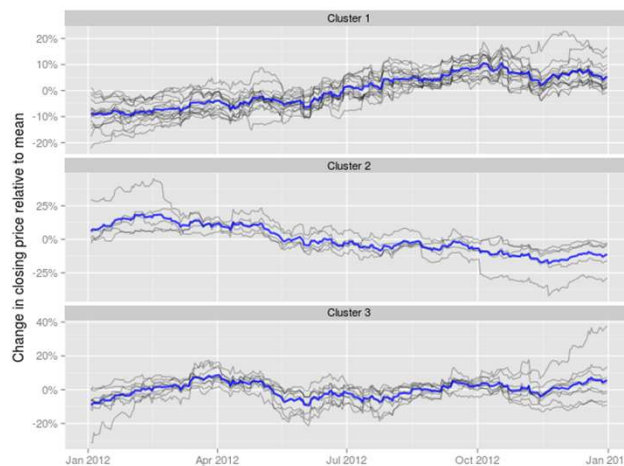
- Very useful for finding groups of different values that might be alternative representations of the same thing.
- This is called *reconciliation* and can be a massive problem for some industries.

First Avenue 35	48 Second Street	Main Boulevard 9
First Avenue 53	Main Blvd 19	Mn Boulevard 11
45 Second Street E	45 Second St	First Ave 35
Furst Avenue 33	44 second street	Main Boulevrd 1
1st Avenue 53	Second Street, 48	Main Bulevard 19

- First Avenue, First Ave, 1st Ave, Furst Ave, etc.

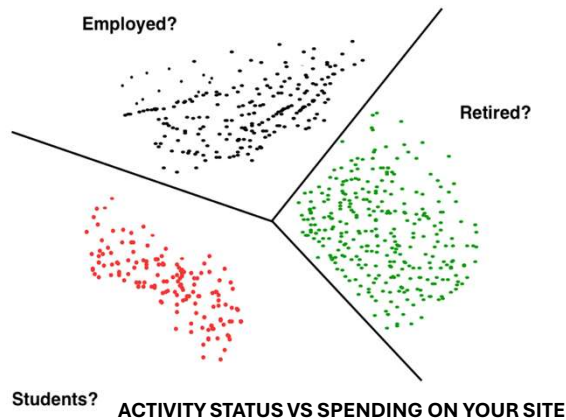
Example: Stocks clustering

- Useful to derive a strategy for assets diversification.



Example: Market segmentation

- Who are my potential customer groups?
 - Who is buying my app and what features should be most appropriate?

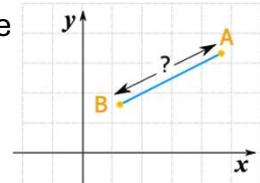


Distance and similarity measures

- Clustering requires being able to compute a distance (or similarity) between the data points.
 - E.g., how similar are two stocks?
- A distance is any function, d , that takes two points (x and y) and returns a scalar value that is a measure for how different these points are.
 - The more dissimilar the points, the larger the distance.
 - In some cases, it may be easier to make more sense to compute a similarity using a function s .
 - $s(x, y)$ can be easily converted into $d(x, y)$.
- While data points do not have to be embedded into a geometric space, it helps to think about their distance (or similarity) as if they were.

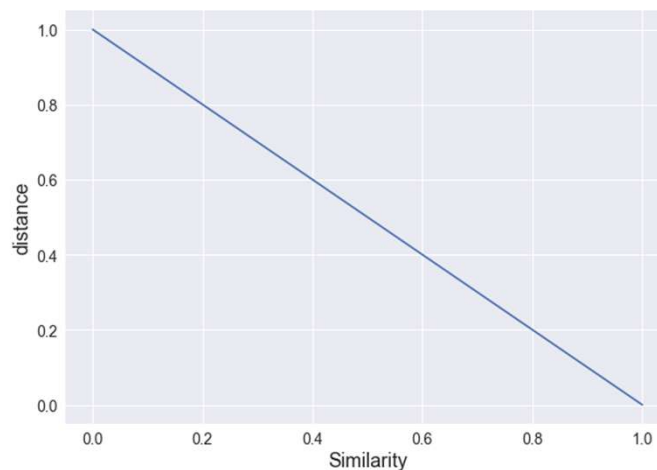
Distance and similarity measures (cont'd.)

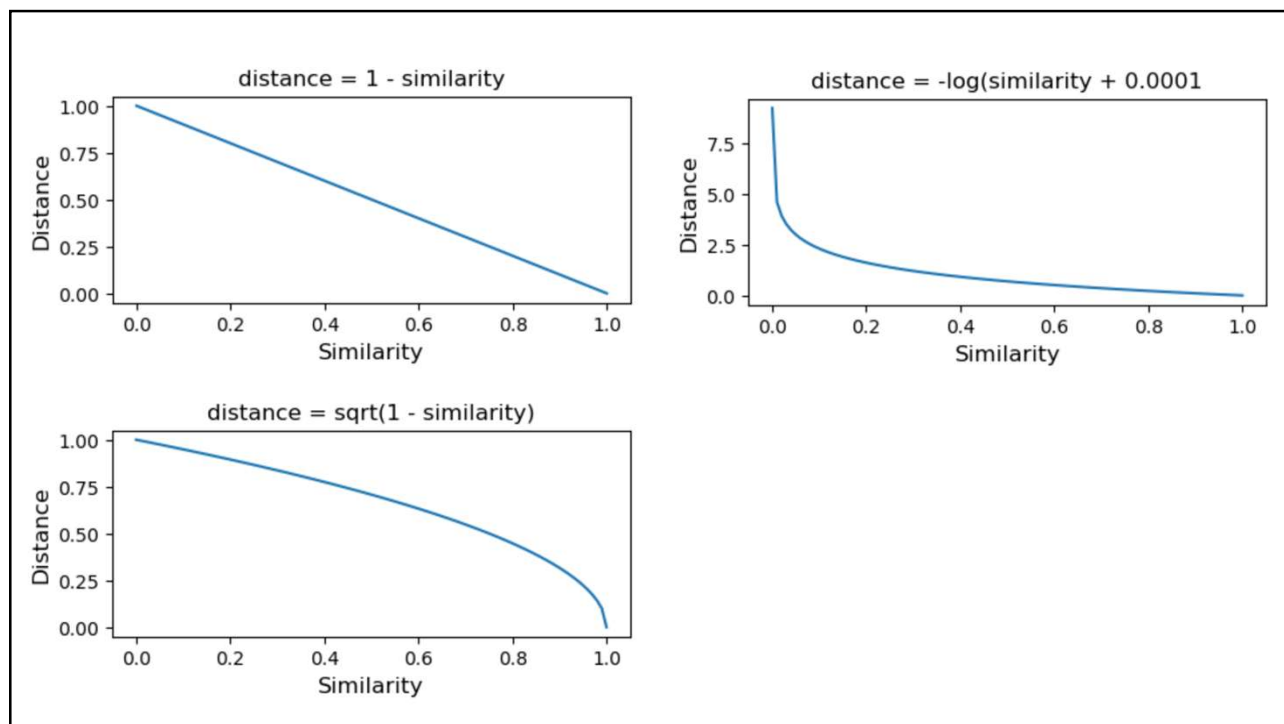
- Sometimes, the concept of distance in geometric space is clear.
 - Ex. Use any measure that considers the locations of the points in such a space.
 - Most points are not just in 2-D but exist in higher-dimensional space.
- Other times, there is no obvious choice as to how to compute the distance between two points.
 - Ex. You are clustering music genres. How do you compute the distance between two songs?



Similarity to distance

- How can you convert a normalized similarity into a distance?



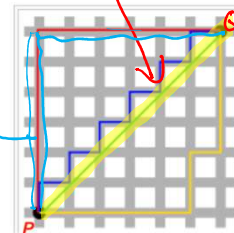


Common distance metrics

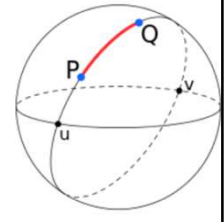
- For simplicity, we'll classify the distances into two broad categories.
 - Euclidean and spherical geometry distances
 - Use the location of the points in a Euclidean or spherical coordinate systems to estimate their distance.
 - Other: Non-Euclidean and non-spherical geometry distances
 - Use properties of points to estimate their distance.
- This is a coarse and somewhat subjective categorization, and other classifications are possible.

Euclidean and spherical distances

- The Euclidean distance
 - Most commonly used distance measure
 - Distance between two points P and Q with coordinate (p_1, p_2) and (q_1, q_2) respectively is: $d_{PQ} = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$
- The Manhattan (or taxicab or cityblock) distance: $d_{PQ} = |p_1 - q_1| + |p_2 - q_2|$
- Geodesic distance
 - Shortest path between two points measured along a surface or curved space, rather than through the surrounding space.



Euclidean vs. Manhattan



Geodesic

Non-Euclidean distance: Edit distance

- Edit distance or minimum number of single-character insertions, deletions, or substitutions to change one string into another.
 - Example: What is the distance between $x = \text{AGACGTAG}$ and $y = \text{GTTTCAGA}$.

$x: \text{A G A C G T A G}$
 $y: \text{G T T C A G A}$

→
 $\begin{matrix} \text{del} & & \text{ins} & & \text{ins} \\ \text{A} & \text{G A} & - & \text{C} & - & \text{G T A G} \\ & \downarrow & & & & \\ - & \text{G T T C A G} & - & \text{A} & & \end{matrix}$

Delete A, substitute A with T, ...
The distance between `x` and `y` is 6

Non-Euclidean distance: Jaccard distance

- Jaccard distance is used with sets or binary attributes.
- It computes the proportion of shared items relative to all items in the sets (Jaccard index), $\frac{|x \cap y|}{|x \cup y|}$. Then, $d_{jaccard} = 1 - \text{Jaccard index}$.

	Muscle Cramps	Mouth Sores	Easy Bruising	Eye Twitch	Metallic Taste
<i>Patient_x</i>	1	1	0	1	0
<i>Patient_y</i>	1	0	0	1	1

$$\text{Jaccard_index}(\text{Patient_x}, \text{Patient_y}) = \frac{|\{\text{Muscle Cramps, Eye Twitch}\}|}{|\{\text{Muscle Cramps, Mouth Sores, Eye Twitch, Metallic Taste}\}|}$$

$$d_{jaccard}(\text{Patient_x}, \text{Patient_y}) = 1 - \text{Jaccard_index} = 1 - 0.5 = 0.5$$

```
def generate_random_string(length, n):
    return [''.join(random.choices(string.ascii_letters +
                                    string.digits, k=length)) for _ in range(n)]

symptoms = ["Muscle Cramps", "Weight Gain", "Easy Bruising", "Metallic Taste", "Paranoia",
            "Leg Pain", "Gas and Bloating", "Mouth Sores", "Nausea, Upset Stomach",
            "Rectal Bleeding", "Shortness of Breath", "Muscle Cramps", "Urine Odor",
            "Swollen Ankles and Feet", "Joint Cracking", "Eye Twitch", "Dry Skin"]

patient_ids = generate_random_string(6, 20)
print(patient_ids)

['zzMuv4', 'HOYLzU', 'F04J3s', 'U1lUcu', 'SjxYyw', '87Pvf3', 'htCfbU', 'aCdRzn', 'aSp7hE', 'qX4NbP',
'xLqUEU', 'CRxtzW', 'PS9N7H', 'IgRE1D', 'zB0wLs', 'YF2SsF', 'ti8lfY', 'BWNbeU', 'ZZBui1', 'p3rJWN']

patient_outcomes = np.random.choice([0, 1], size=(len(symptoms)))
patient_outcomes

array([0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0])
```

```
data = [np.random.choice([0, 1], size=(len(symptoms))) for _ in range(len(patient_ids))]
data
```

```
[array([1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1]),
 array([1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0]),
 array([1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0]),
 array([0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0]),
 array([1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1]),
 array([1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1]),
 array([0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0]),
 array([0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0]),
 array([1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1]),
 array([1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0]),
 array([1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0]),
 array([0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1]),
 array([1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1]),
 array([1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1]),
 array([0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0]),
 array([1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0]),
 array([0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0]),
 array([0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0]),
 array([1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1]),
 array([0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1])]
```

20x17.

```
patients_data = pd.DataFrame(columns=symptoms, data=data, index=patient_ids)
patients_data.head()
```

	Muscle Cramps	Weight Gain	Easy Bruising	Metallic Taste	Paranoia	Leg Pain	Gas and Bloating	Mouth Sores	Nausea, Upset Stomach	Rectal Bleeding	Shortness of Breath	Muscle Cramps	Urine Odor	Swollen Ankles and Feet	Joint Cracking	Eye Twitch	Dry Skin
zzMuv4	1	1	0	1	0	1	0	0	1	0	0	0	0	0	1	1	1
HOYLzU	1	1	1	1	1	0	0	0	0	1	1	1	1	0	1	0	0
F04J3s	1	1	1	0	1	1	1	0	1	1	1	1	1	0	0	1	0
U1IUcu	0	0	1	0	1	0	1	0	1	1	0	1	0	0	0	1	0
SjxYyw	1	0	1	1	0	1	1	0	1	1	1	1	1	0	0	1	1

```
patients_data.shape
```

(20, 17)

```
def compute_Jaccard_distance(vec_1, vec_2):

    # All the symptoms shared by both
    intersection = sum((vec_1 == vec_2) & (vec_1 == 1))

    # All the symptoms that either one has
    union = sum((vec_1 == 1) | (vec_2 == 1))

    return 1 - intersection / union

vec_1 = np.array([1, 0, 1, 1, 1])
vec_2 = np.array([1, 0, 0, 1, 1])

print(compute_Jaccard_distance(vec_1, vec_2))
```

0.25

$\frac{3}{4} = 0.75$

Distance matrix

- It is common to store the pairwise distances between data points in a distance matrix M .
- The matrix has dimensions $n \times n$, where n is the number of data points.
- For any two points p_i and p_j , it is typically assumed that:

$$M[i, j] = d(p_i, p_j)$$

$$M[j, i] = d(p_j, p_i)$$

where $d(p_i, p_j)$ is the distance between points p_i and p_j , and the matrix M is symmetric.

```

# Compute all pair-wise Jaccard coefficients
dist_patients = np.zeros([len(patient_ids), len(patient_ids)])

for i in range(patients_data.shape[0]):
    for j in range(i, patients_data.shape[0]):
        vec_1 = patients_data.iloc[i]
        vec_2 = patients_data.iloc[j]

        # compute distance and round to two decimal points
        dist_patients[i, j] = round(compute_Jaccard_distance(vec_1, vec_2), 2)
        dist_patients[j, i] = dist_patients[i, j]

dist_patients.shape

(20, 20)

```

dist_patients

```

array([[0.  , 0.57, 0.5  , 0.58, 0.64, 0.85, 0.5  , 0.4  , 0.62, 0.77, 0.73,
        0.75, 0.5  , 0.81, 0.57, 0.67, 0.5  , 0.5  , 0.42, 0.64],
       [0.57, 0.  , 0.57, 0.64, 0.5  , 0.94, 0.47, 0.62, 0.46, 0.71, 0.67,
        0.69, 0.57, 0.69, 0.62, 0.71, 0.46, 0.27, 0.6  , 0.79],
       [0.5  , 0.57, 0.  , 0.69, 0.42, 0.75, 0.5  , 0.67, 0.62, 0.86, 0.83,
        0.75, 0.71, 0.64, 0.57, 0.67, 0.5  , 0.5  , 0.64, 0.64],
       [0.58, 0.64, 0.69, 0.  , 0.71, 0.83, 0.46, 0.75, 0.79, 0.85, 0.56,
        0.6  , 0.45, 0.62, 0.64, 0.75, 0.69, 0.47, 0.8  , 0.6  ],
       [0.64, 0.5  , 0.42, 0.71, 0.  , 0.77, 0.53, 0.79, 0.42, 0.69, 0.64,
        ...,
       [0.42, 0.6  , 0.64, 0.8  , 0.67, 0.67, 0.53, 0.3  , 0.64, 0.58, 0.75,
        0.86, 0.64, 0.57, 0.6  , 0.58, 0.64, 0.53, 0.  , 0.67],
       [0.64, 0.79, 0.64, 0.6  , 0.67, 0.8  , 0.5  , 0.56, 0.75, 0.7  , 0.78,
        0.91, 0.64, 0.67, 0.79, 0.92, 0.85, 0.69, 0.67, 0.  ]])

```

20x20

```

from numpy import unravel_index

max_idx = unravel_index(np.argmax(dist_patients), dist_patients.shape)

print(np.nanmax(dist_patients), np.nanargmax(dist_patients), *max_idx)

patients_data.iloc[[max_idx[0], max_idx[1]]]

```

0.94 25 1 5)

	Muscle Cramps	Weight Gain	Easy Bruising	Metallic Taste	Paranoia	Leg Pain	Gas and Bloating	Mouth Sores	Nausea, Upset Stomach	Rectal Bleeding	Shortness of Breath	Muscle Cramps	Urine Odor	Swollen Ankles and Feet	Joint Cracking	Eye Twitch	Dry Skin
lumtUf	1	1	0	1	1	0	0	1	0	1	1	1	1	1	0	1	0
qScZ1p	0	1	0	0	0	1	1	0	1	0	0	0	0	0	1	0	1

$$J_d = 1 - \frac{1}{16} = 0.94$$

```

dist_patients_temp = dist_patients.copy()

for i in range(patients_data.shape[0]):
    dist_patients_temp[i, i] = 1000

min_idx = unravel_index(np.argmin(dist_patients_temp), dist_patients_temp.shape)

print(np.nanmin(dist_patients_temp), np.nanargmin(dist_patients_temp), *min_idx)

patients_data.iloc[[min_idx[0], min_idx[1]]]

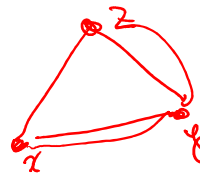
```

0.27 37 1 17

	Muscle Cramps	Weight Gain	Easy Bruising	Metallic Taste	Paranoia	Leg Pain	Gas and Bloating	Mouth Sores	Nausea, Upset Stomach	Rectal Bleeding	Shortness of Breath	Muscle Cramps	Urine Odor	Swollen Ankles and Feet	Joint Cracking	Eye Twitch	Dry Skin
lumtUf	1	1	0	1	1	0	0	1	0	1	1	1	1	1	0	1	0
plr7NJ	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1

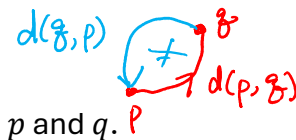
Devising your own distance

- There are certain properties that a distance (or similarity) function should have.
 - A distance in a mathematical sense must have at least the following properties (properties of a metric).
 - **Non-negativity** rule: $d(x, y) \geq 0$
 - **Symmetry** rule: $d(x, y) = d(y, x)$
 - **Triangle Inequality** rule: $d(x, y) + d(y, z) \geq d(x, z)$



Devising your own distance (cont'd.)

- In practice, it may be hard to identify a distance that guarantees all these properties.
 - Some metrics are not symmetric.
 - E.g., length of the shortest clockwise path between p and q .
- For technical reasons, the symmetry property is usually highly desirable.
 - You can always construct a symmetric distance function from an asymmetric one:



$$dS(x, y) = \frac{d(x, y) + d(y, x)}{2}$$

Clustering approaches

In this class we will cover two approaches to clustering a set of points into clusters.

- Hierarchical
 - Each point is a cluster.
 - Repeatedly combine the two “closest” clusters into one.
 - Stop when only one cluster remains.
 - This is also known as “agglomerative hierarchical clustering”.
- Point-to-cluster assignment
 - Maintain a set of k clusters (e.g., $k = 3$).
 - Place points into “closest” cluster.
 - Iteratively repeat the process until the solution is stable.

Clustering approaches (cont'd.)

- There are more advanced/complex approaches.
 - Difficult to combine into categories because of the diversity of approaches and overlaps.
 - Probabilistic methods increasingly popular.

(Agglomerative) bottom-up
Hierarchical clustering

